

Code:

```
#include <iostream>

#include <fstream>

#include <vector>

#include <string>

#include <algorithm>

#include <regex>

#include <conio.h> // For _getch()


using namespace std;


// ===== Base User Class =====

class User {

protected:

    string id, password;

public:

    virtual void login() = 0;


    string getID() const { return id; }

    string getPassword() const { return password; }


    string getPasswordInput() {

        string pass;

        char ch;

        while ((ch = _getch()) != '\r') {

            if (ch == '\b') {

                if (!pass.empty()) {

                    pass.pop_back();

                }

            }

            pass += ch;

        }

        return pass;

    }

};
```

```

        cout << "\b \b";

    }

    } else {

        pass += ch;

        cout << '*';

    }

}

cout << endl;

return pass;

}

virtual ~User() {}

};

// ===== Student Class =====

class Student : public User {

public:

    string firstName, lastName, department, semester;

    float gpa;

    Student() {}

    Student(string cmsID, string fName, string lName, string dept, string sem, float g,
string pass) {

        this->id = cmsID;

        this->firstName = fName;

        this->lastName = lName;

        this->department = dept;

```

```

        this->semester = sem;

        this->gpa = g;

        this->password = pass;
    }

    void display() const {

        cout << "CMS ID: " << id << "\nName: " << firstName << " " << lastName
            << "\nDepartment: " << department << "\nSemester: " << semester
            << "\nGPA: " << gpa << "\n-----\n";

    }

    void setPassword(const string& newPass) {

        password = newPass;

    }

    void login() override {

        cout << "Enter CMS ID (numeric only): ";

        cin >> id;

        while (!regex_match(id, regex("[0-9]+$"))) {

            cout << "Invalid CMS ID. Enter a positive number only: ";

            cin >> id;

        }

        cout << "Enter Password: ";

        password = getPasswordInput();

    }

};

// ===== Teacher Class =====

```

```

class Teacher : public User {
public:
    string name;

    Teacher() {}

    Teacher(string id, string name, string pass) {
        this->id = id;
        this->name = name;
        this->password = pass;
    }

    void login() override {
        cout << "Enter Teacher ID: "; cin >> id;
        cout << "Enter Password: ";
        password = getPasswordInput();
    }
};

// ===== Admin Class =====

class Admin : public User {
public:
    Admin() {
        id = "admin";
        password = "Admin@123";
    }

    void login() override {

```

```

string inputID, inputPass;

cout << "Enter Admin ID: "; cin >> inputID;

cout << "Enter Password: ";

inputPass = getPasswordInput();


if (inputID != id || inputPass != password) {
    cout << "Invalid credentials!\n";
    throw runtime_error("Login Failed");
}
}

};


// ===== Globals =====

vector<Student> students;
vector<Teacher> teachers;


// ===== Utility Functions =====

bool isValidPassword(const string& pass) {
    return pass.length() >= 7 &&
        regex_search(pass, regex("[0-9]")) &&
        regex_search(pass, regex("[^A-Za-z0-9]"));
}

bool isCMSIDValid(const string& id) {
    return regex_match(id, regex("[0-9]+$"));
}

void loadStudents() {

```

```

students.clear();

ifstream in("students.txt");

string cms, fname, lname, dept, sem, pass;

float gpa;

while (in >> cms >> fname >> lname >> dept >> sem >> gpa >> pass) {
    students.emplace_back(cms, fname, lname, dept, sem, gpa, pass);
}

in.close();
}

void saveStudents() {
    ofstream out("students.txt");

    for (auto& s : students) {
        out << s.getID() << ' ' << s.firstName << ' ' << s.lastName << ' '
            << s.department << ' ' << s.semester << ' ' << s.gpa << ' ' << s.getPassword() << '\n';
    }

    out.close();
}

void loadTeachers() {
    teachers.clear();

    ifstream in("teachers.txt");

    string id, name, pass;

    while (in >> id >> name >> pass) {
        teachers.emplace_back(id, name, pass);
    }
}

```

```
        in.close();
    }

void saveTeachers() {
    ofstream out("teachers.txt");
    for (auto& t : teachers) {
        out << t.getID() << ' ' << t.name << ' ' << t.getPassword() << '\n';
    }
    out.close();
}
```

```
// ===== Admin Portal =====
```

```
void adminPortal() {
    Admin admin;
    try {
        admin.login();
    } catch (...) {
        return;
    }
}
```

```
int choice;
do {
    cout << "\n--- Admin Portal ---"
        << "\n1. Add Teacher"
        << "\n2. Add Student"
        << "\n3. Delete Teacher"
        << "\n4. Delete Student"
        << "\n5. View All Teachers"
```

```

    << "\n6. View All Students"

    << "\n7. Back"

    << "\nChoice: ";

cin >> choice;


if (choice == 1) {
    string id, name, pass;

    cout << "Enter Teacher ID: "; cin >> id;

    cout << "Enter Teacher Name: "; cin >> name;

    do {
        cout << "Enter Password: ";

        pass = admin.getPasswordInput();

    } while (!isValidPassword(pass));

    if (any_of(teachers.begin(), teachers.end(), [&](Teacher& t){ return t.getID() == id;
})) {

        cout << "Teacher already exists.\n";

    } else {

        teachers.emplace_back(id, name, pass);

        saveTeachers();

        cout << "Teacher added.\n";

    }

} else if (choice == 2) {

    string cms, fname, lname, dept, sem, pass;

    float gpa;

    do {

        cout << "Enter CMS ID (positive numeric only): ";

```

```

        cin >> cms;

    } while (!isCMSIDValid(cms));

    if (any_of(students.begin(), students.end(), [&](Student& s){ return s.getID() ==
cms; }))) {

        cout << "Student already exists.\n";

    } else {

        cout << "Enter First Name: "; cin >> fname;

        cout << "Enter Last Name: "; cin >> lname;

        cout << "Enter Department: "; cin >> dept;

        cout << "Enter Semester: "; cin >> sem;

        do {

            cout << "Enter GPA (positive only): ";

            cin >> gpa;

            if (gpa <= 0) cout << "GPA must be positive.\n";

        } while (gpa <= 0);

        do {

            cout << "Enter Password: ";

            pass = admin.getPasswordInput();

        } while (!isValidPassword(pass));

        students.emplace_back(cms, fname, lname, dept, sem, gpa, pass);

        saveStudents();

        cout << "Student added.\n";

    }

} else if (choice == 3) {

```

```

string id;

cout << "Enter Teacher ID to delete: "; cin >> id;

auto it = remove_if(teachers.begin(), teachers.end(), [&](Teacher& t){ return
t.getID() == id; });

if (it != teachers.end()) {

    teachers.erase(it, teachers.end());

    saveTeachers();

    cout << "Teacher deleted.\n";

} else cout << "Not found.\n";


} else if (choice == 4) {

    string cms;

    cout << "Enter Student CMS ID to delete: "; cin >> cms;

    auto it = remove_if(students.begin(), students.end(), [&](Student& s){ return
s.getID() == cms; });

    if (it != students.end()) {

        students.erase(it, students.end());

        saveStudents();

        string filename = cms + "_data.txt";

        remove(filename.c_str());

        cout << "Student and all associated data deleted.\n";

    } else cout << "Not found.\n";


} else if (choice == 5) {

    cout << "\n--- List of Teachers ---\n";

    if (teachers.empty()) {

        cout << "No teacher records available.\n";

    } else {

        for (auto& t : teachers) {

```

```

        cout << "ID: " << t.getID() << " | Name: " << t.name << '\n';
    }
}

} else if (choice == 6) {
    cout << "\n--- List of Students ---\n";

    if (students.empty()) {
        cout << "No student records available.\n";
    } else {
        for (auto& s : students) {
            s.display();
        }
    }
}

} while (choice != 7);
}

// ===== Teacher Portal =====

void teacherPortal() {
    Teacher temp;

    temp.login();

    auto it = find_if(teachers.begin(), teachers.end(),
        [&](Teacher& t){ return t.getID() == temp.getID() && t.getPassword() ==
temp.getPassword(); });

    if (it == teachers.end()) {
        cout << "Invalid login!\n";

        return;
    }
}

```

```
}
```

```
int choice;
```

```
do {
```

```
    cout << "\n--- Teacher Portal ---\n1. Edit Student Record\n2. View All Students\n3. Back\nChoice: ";
```

```
    cin >> choice;
```

```
    if (choice == 1) {
```

```
        string cms;
```

```
        cout << "Enter Student CMS ID to edit: ";
```

```
        cin >> cms;
```

```
        auto sit = find_if(students.begin(), students.end(), [&](Student& s){ return s.getID() == cms; });
```

```
        if (sit == students.end()) {
```

```
            cout << "Student not found.\n";
```

```
        } else {
```

```
            cout << "Editing Record\n";
```

```
            cout << "First Name (" << sit->firstName << "): "; cin >> sit->firstName;
```

```
            cout << "Last Name (" << sit->lastName << "): "; cin >> sit->lastName;
```

```
            cout << "Department (" << sit->department << "): "; cin >> sit->department;
```

```
            cout << "Semester (" << sit->semester << "): "; cin >> sit->semester;
```

```
            do {
```

```
                cout << "GPA (" << sit->gpa << "): "; cin >> sit->gpa;
```

```
                if (sit->gpa <= 0) cout << "GPA must be positive.\n";
```

```
            } while (sit->gpa <= 0);
```

```
            saveStudents();
```

```
            cout << "Record updated.\n";
```

```
        }
```

```

    } else if (choice == 2) {
        if (students.empty()) {
            cout << "No student records available.\n";
        } else {
            for (auto& s : students) s.display();
        }
    }
} while (choice != 3);
}

// ===== Student Portal =====

void studentPortal() {
    Student temp;
    temp.login();

    auto it = find_if(students.begin(), students.end(),
        [&](Student& s){ return s.getID() == temp.getID() && s.getPassword() ==
temp.getPassword(); });
    if (it == students.end()) {
        cout << "Invalid login!\n";
        return;
    }

    int choice;
    do {
        cout << "\n--- Student Portal ---\n1. View Info\n2. Update Name\n3. Update
Password\n4. Back\nChoice: ";
        cin >> choice;
    } while (choice < 1 || choice > 4);
}

```

```

    if (choice == 1) {
        it->display();
    } else if (choice == 2) {
        cout << "New First Name: "; cin >> it->firstName;
        cout << "New Last Name: "; cin >> it->lastName;
        saveStudents();
        cout << "Name updated.\n";
    } else if (choice == 3) {
        string newPass;
        do {
            cout << "Enter New Password: ";
            newPass = temp.getPasswordInput();
        } while (!isValidPassword(newPass));
        it->setPassword(newPass);
        saveStudents();
        cout << "Password updated.\n";
    }
} while (choice != 4);
}

```

// ===== Main =====

```

int main() {
    loadStudents();
    loadTeachers();

    int choice;
    do {

```

```

        cout << "\n=== Main Menu ===\n1. Admin\n2. Teacher\n3. Student\n4.
Exit\nChoice: ";

        cin >> choice;

        switch (choice) {

            case 1: adminPortal(); break;

            case 2: teacherPortal(); break;

            case 3: studentPortal(); break;

            case 4: cout << "Goodbye!\n"; break;

            default: cout << "Invalid option!\n";

        }

    } while (choice != 4);

    return 0;
}

```

OUTOUT:

```

=== Main Menu ===
1. Admin
2. Teacher
3. Student
4. Exit
Choice: 1_

```

```

Enter Admin ID: admin
Enter Password: *****

--- Admin Portal ---
1. Add Teacher
2. Add Student
3. Delete Teacher
4. Delete Student
5. View All Teachers
6. View All Students
7. Back
Choice: _

```

Admin Password: Admin@123

```
--- Admin Portal ---
1. Add Teacher
2. Add Student
3. Delete Teacher
4. Delete Student
5. View All Teachers
6. View All Students
7. Back
Choice: 1
Enter Teacher ID: 4567
Enter Teacher Name: Sulaman
Enter Password: *****
Teacher added.
```

```
--- Admin Portal ---
1. Add Teacher
2. Add Student
3. Delete Teacher
4. Delete Student
5. View All Teachers
6. View All Students
7. Back
Choice: _
```

```
--- Admin Portal ---
1. Add Teacher
2. Add Student
3. Delete Teacher
4. Delete Student
5. View All Teachers
6. View All Students
7. Back
Choice: 2
Enter CMS ID (positive numeric only): 2742
Enter First Name: Umair
Enter Last Name: Rana
Enter Department: Fest
Enter Semester: 2nd
Enter GPA (positive only): 3.02
Enter Password: *****
Student added.
```

```
=== Main Menu ===
1. Admin
2. Teacher
3. Student
4. Exit
Choice: 2
Enter Teacher ID: 1234
Enter Password: *****
---
```

```
Teacher Portal ---
1. Edit Student Record
2. View All Students
3. Back
Choice: 
```

```
Choice: 1
Enter Student CMS ID to edit: 2742
Editing Record
First Name (Umair): Umair
Last Name (Rana): Rana
Department (Fest):
fest
Semester (2nd): 3rd
GPA (3.02): 3.2
Record updated.
---
```

```
Teacher Portal ---
1. Edit Student Record
2. View All Students
3. Back
Choice:
```

```
=== Main Menu ===
1. Admin
2. Teacher
3. Student
4. Exit
Choice: 3
Enter CMS ID (numeric only): 2742
Enter Password: *****
---
```

```
Student Portal ---
1. View Info
2. Update Name
3. Update Password
4. Back
Choice:
```

--- Student Portal ---

1. View Info
2. Update Name
3. Update Password
4. Back

Choice: 1

CMS ID: 2742

Name: Umair Rana

Department: fest

Semester: 3rd

GPA: 3.2

--- Student Portal ---

1. View Info
2. Update Name
3. Update Password
4. Back

Choice: 4

=== Main Menu ===

1. Admin
2. Teacher
3. Student
4. Exit

Choice: 4

Goodbye!

Process returned 0 (0x0) execution time : 494.842 s
Press any key to continue.